

CR-001 – Change Implementation Report

1. Change Request Details

Change Request ID: CR-001

Title: Advanced Search, Course Filtering Improvements, and Responsive Card Grid Fix

Priority: Medium

Difficulty: Easy

Change Type: Frontend Change

Implementation Status: Implemented

Business Requirement

Students and visitors should be able to find suitable courses more easily across any device. The existing public course catalogue provided only basic searching by title and category and did not provide sufficient filtering options. Furthermore, when search or filter criteria resulted in few matching courses, the course card grid layout distorted by stretching isolated cards across the entire container width, and filter controls lacked responsive adaptation on tablet and mobile viewports.

Existing Behaviour

The public course catalogue previously supported only basic searching by course title and category. It did not support:

- Level filtering (Beginner, Intermediate, Advanced)
- Searching within course descriptions
- Searching by duration
- Minimum and maximum price filtering
- Clearing all filters at once
- Displaying active filters as chips
- Displaying the count of matching results out of total courses
- Separate messages for an empty catalogue versus no matching results
- Responsive card grid behavior (using auto-fit caused single or few cards to over-stretch across the full container width)
- Multi-device responsive filter layout for tablet and mobile viewports

Requested Change

The public course catalogue must be enhanced with advanced frontend search and multi-criteria filtering functionality, along with responsive layout improvements for course cards and filter controls. Users should be able to search and filter courses using multiple criteria simultaneously with strict AND logic, safely handle null data, and view a well-proportioned card grid on any screen size.

2. Change Analysis

2.1 Objective of the Change

The objective of CR-001 is to improve the usability and visual quality of the public course catalogue by allowing students and visitors to quickly locate suitable courses based on:

- Course title
- Category
- Level
- Description
- Duration
- Minimum price
- Maximum price

All selected filters must work together using **AND** logic.

For example, if a user selects:

Category = Programming

Level = Advanced

Minimum Price = 100

the catalogue should display only courses that satisfy all selected conditions simultaneously.

In addition, course cards must render in a stable responsive grid (using `auto-fill`) so that individual cards maintain uniform card dimensions without stretching unnaturally across the screen when few courses match, and filter controls must adapt fluidly between desktop, tablet, and mobile layouts.

3. Impact Analysis

Area	Impact	Details
Frontend	Yes	Advanced search, multi-criteria filtering, active filter chips, dynamic clear button, result counters, and empty states added to the course catalogue.
Backend	No	Existing backend functionality remains unchanged.
Database	No	No database structure or data changes are required.
API	No	No additional API request or API response changes are required. Existing endpoint (<code>GET /courses</code>) is utilized.
Authentication	No	The feature works for both public (logged-out) visitors and authenticated users.
Styling	Yes	Updated <code>.course-grid</code> with <code>auto-fill</code> , added <code>.filter-bar</code> grid, number input spinner suppression, <code>.filter-chip</code> badges, <code>.clear-filter-btn</code> , and media queries for tablet ($\leq 1100\text{px}$) and mobile ($\leq 700\text{px}$).
Performance	Low	Filtering is executed in-memory on the client against already-fetched course data with minimal overhead.

3.1 Affected Files

Primary Files:

- `frontend/src/pages/Courses.jsx`
- `frontend/src/App.css`

Backend Files:

No backend files need to be modified.

Database:

No database changes are required.

API:

No API changes are required.

4. Risk Analysis

Risk	Impact	Mitigation
Existing catalogue functionality could be affected	Medium	Existing course retrieval, card rendering, and routing tested thoroughly after implementation.
Null or undefined descriptions could cause search errors	Medium	Use null-safe string conversion (<code>String(course.description "")</code>) before executing <code>.toLowerCase().includes()</code> .
Null or numeric duration values could cause runtime errors	Medium	Safely convert duration to string (<code>String(course.duration "")</code>) before evaluation.
Incorrect AND filtering could display unwanted courses	Medium	Strictly combine all criteria predicates with Boolean <code>&&</code> logic in the filter loop.
Price fields could accept invalid or negative values	Low	Set <code>min="0"</code> on price inputs and convert with <code>Number()</code> , safely ignoring empty strings and invalid numbers.
Clear All may fail to reset all active filters	Medium	Reset all 5 filter states (<code>searchText</code> , <code>selectedCategory</code> , <code>selectedLevel</code> , <code>minPrice</code> , <code>maxPrice</code>) synchronously in <code>clearAllFilters()</code> .
Course cards over-stretching across full width when few results match	Low	Replace CSS <code>grid-template-columns: repeat(auto-fit, ...)</code> with <code>repeat(auto-fill, minmax(270px, 1fr))</code> so columns retain fixed minimum width.
Filter controls overflowing or breaking on smaller screen viewports	Low	Implement responsive CSS media queries for $\leq 1100\text{px}$ (3 columns) and $\leq 700\text{px}$ (single-column vertical stack).

5. Functional Requirements

The implementation must satisfy the following functional and visual requirements:

5.1 Text Search

Add a comprehensive text search field to the public course catalogue. The search must match across:

- Course title
- Category
- Level
- Description
- Duration

The search must be case-insensitive and handle whitespace cleanly via `.trim().toLowerCase()`.

5.2 Level Filter

Add a Level dropdown selector containing:

- All Levels (value: "All")
- Beginner
- Intermediate
- Advanced

Selecting a level must display only courses matching that exact level.

5.3 Price Range

Add numeric price boundary inputs:

- **Minimum Price:** Filters out courses with price lower than the input.
- **Maximum Price:** Filters out courses with price higher than the input.

The catalogue should display only courses whose numeric price falls within the specified range.

5.4 Combined Filters

All filters must work together using **AND** logic. For example:

Search: Java

Category: Programming

Level: Advanced

Minimum Price: 100

The displayed courses must satisfy all four conditions simultaneously.

5.5 Result Counter

Display the count of currently matched courses relative to total available courses.

Example:

Showing 4 of 8 courses

Where:

- 4 = number of courses currently matching all active filters
- 8 = total number of courses loaded in the catalogue

5.6 Active Filter Chips

Display visual chips for all currently active filters to give users clear feedback.

Examples:

- Search: Backend
- Category: Programming
- Level: Advanced
- Min price: 100
- Max price: 500

5.7 Clear All Filters

Provide a dedicated "Clear All Filters" button. The button must reset:

- Search text → ""
- Category → "All"
- Level → "All"

- Minimum price → ""
- Maximum price → ""

The Clear All button should only be rendered when at least one filter is active. After clearing, all original courses must be immediately restored.

5.8 Empty States

The system must distinguish between an empty database and an unmatched search by providing two distinct empty-state messages:

1. No Courses Exist:

When the catalogue contains zero courses from the backend, display:

"There are no courses available at the moment."

2. No Matching Courses:

When courses exist in the catalogue but none match the applied filter criteria, display:

"No courses match your filters. Try changing or clearing your search criteria."

5.9 Null Safety

The application must never crash or throw runtime exceptions when course data contains null or undefined values:

- null or undefined description → treated safely as empty string ("")
- null or undefined duration → treated safely as empty string ("")
- null or non-numeric price → safely evaluated with `Number.isNaN()`

5.10 Responsive Card Grid Layout (Cards Fix)

The course card grid must render cleanly without layout degradation:

- When many courses are displayed, cards fill the grid rows evenly.
- When filtering narrows down the catalogue to 1 or 2 courses, the cards must **not** over-expand or stretch across the entire screen.
- Grid layout must use `repeat(auto-fill, minmax(270px, 1fr))` so empty column tracks remain unfilled rather than stretching existing cards.

5.11 Responsive Filter Controls

The filter controls must adapt smoothly across different viewport sizes:

- **Desktop (> 1100px):** 6-column grid layout (Search, Category, Level, Min Price, Max Price, Clear button).
- **Tablet (≤ 1100px):** 3-column grid layout for ergonomic touch targets.
- **Mobile (≤ 700px):** Single-column stacked layout with full-width Clear button and vertical filter chips.

6. Implementation

6.1 Frontend Implementation

The advanced filtering functionality and responsive controls were implemented in:

`frontend/src/pages/Courses.jsx`

The following functionality was added:

- Search text input supporting multi-field matching across title, category, level, description, and duration
- Dynamic category dropdown extracted safely from loaded course data using `Set` and `filter(Boolean)`
- Level dropdown filter (All Levels, Beginner, Intermediate, Advanced)
- Minimum price input with numeric validation
- Maximum price input with numeric validation
- Strict combined **AND** logic across all active filter predicates
- Result counter displaying Showing X of Y courses
- Active filter chips rendering current query parameters dynamically
- Conditional "Clear All Filters" button visible only when `hasActiveFilters` is true
- Differentiated empty states for empty catalogue vs. no matching results
- Comprehensive null-safe handling of course titles, categories, levels, descriptions, durations, and prices

6.2 Styling Implementation

Styling changes were made in:

`frontend/src/App.css`

The styling was updated to support:

- **Course grid layout fix:** Updated `.course-grid` to `grid-template-columns: repeat(auto-fill, minmax(270px, 1fr));` to prevent card stretching when few results match
- **Filter bar grid:** Added `.filter-bar` with `display: grid; grid-template-columns: 2fr 1fr 1fr 1fr 1fr auto; gap: 12px;`
- **Number input styling:** Removed WebKit and Firefox browser spinners on price inputs for clean UI
- **Clear All button:** Added `.clear-filter-btn` styled with danger theme (`#dc3545`) and hover transition
- **Filter chips:** Added `.active-filters` and `.filter-chip` pill styling with soft blue theme (`#e7f1ff`, `#0d6efd`)
- **Tablet breakpoint ($\leq 1100\text{px}$):** Reorganized filter bar into `grid-template-columns: repeat(3, 1fr)`
- **Mobile breakpoint ($\leq 700\text{px}$):** Reorganized filter bar into single-column layout (`1fr`), full-width button, and vertical chip stack

6.3 Backend Implementation

No backend changes were required. The existing backend REST API endpoint `GET /courses` provides complete course objects containing all requisite fields.

The existing course data is retrieved using the existing functionality. Filtering is performed entirely on the frontend.

6.4 Database Implementation

No database changes were required. The existing SQLite course schema already stores `title`, `description`, `category`, `level`, `price`, and `duration`.

6.5 API Implementation

No additional API request was introduced. The feature operates entirely on the course data already available on the frontend.

7. Filtering Logic

The filtering process follows strict **AND** logic. A course is displayed only when it satisfies every active filter condition.

The filtering algorithm executes the following sequence for each course in the loaded dataset:

1. **Text Search:** Checks if `title`, `category`, `level`, `description`, or `duration` contains the search query (case-insensitive, whitespace-trimmed).
2. **Category Filter:** Evaluates if `selectedCategory === "All"` or matches `course.category`.
3. **Level Filter:** Evaluates if `selectedLevel === "All"` or matches `course.level`.
4. **Minimum Price Filter:** If specified, validates that `coursePrice >= Number(minPrice)`.
5. **Maximum Price Filter:** If specified, validates that `coursePrice <= Number(maxPrice)`.

If a filter has not been selected or entered by the user, that specific filter is considered inactive and does not restrict results.

Example:

If a user selects:

- Category: **Programming**
- Level: **Advanced**
- Minimum Price: **100**
- Maximum Price: **500**

only courses satisfying all four conditions simultaneously are rendered in the course grid.

8. Null Handling

The implementation safely handles courses where `description`, `duration`, or other fields are null or undefined. Instead of directly invoking string methods on raw properties, values are coerced to safe strings prior to inspection:

- `const description = String(course.description || "");`

- `const duration = String(course.duration || "");`
- `const title = String(course.title || "");`

This completely prevents runtime errors such as `TypeError: Cannot read properties of null (reading 'toLowerCase')`.

9. Testing Strategy

Testing was performed against all functional requirements, responsive layout criteria, and edge cases defined in CR-001.

Testing coverage includes:

- Functional search and multi-field keyword testing
- Category and level dropdown filter testing
- Price range boundary testing (valid, empty, zero, invalid)
- Combined multi-criteria AND filtering testing
- Dynamic "Clear All Filters" button reset verification
- Differentiated empty-state message verification
- Null and missing attribute data safety testing
- Course card grid responsive layout verification (auto-fill behavior with 1, 2, and multiple cards)
- Responsive filter bar testing across desktop, tablet ($\leq 1100\text{px}$), and mobile ($\leq 700\text{px}$) viewports
- Public / unauthenticated access verification
- Regression testing of course cards, navigation, and course details links

10. Test Cases

Test ID	Test Case	Test Steps	Expected Result	Actual Result	Status
TC-001	Search by course title	Enter a course title in the search field	Matching courses are displayed	Matching courses displayed	Pass
TC-002	Search by category	Enter a category in the search field	Courses belonging to category are displayed	Correct results displayed	Pass
TC-003	Search by level	Enter a level such as "Advanced" in search field	Matching courses are displayed	Correct results displayed	Pass
TC-004	Search by description	Enter text contained only in course description	Courses containing text in description are displayed	Correct results displayed	Pass
TC-005	Search by duration	Enter duration-related text (e.g. "weeks", "hours")	Matching courses are displayed	Correct results displayed	Pass

Test ID	Test Case	Test Steps	Expected Result	Actual Result	Status
TC-006	Select Beginner level	Select "Beginner" from Level dropdown	Only Beginner courses are displayed	Correct results displayed	Pass
TC-007	Select Intermediate level	Select "Intermediate" from Level dropdown	Only Intermediate courses are displayed	Correct results displayed	Pass
TC-008	Select Advanced level	Select "Advanced" from Level dropdown	Only Advanced courses are displayed	Correct results displayed	Pass
TC-009	Select All level	Select "All Levels" from Level dropdown	Courses of all levels are displayed	All applicable courses displayed	Pass
TC-010	Apply minimum price	Enter a minimum price in Min Price input	Courses below minimum price are excluded	Correct results displayed	Pass
TC-011	Apply maximum price	Enter a maximum price in Max Price input	Courses above maximum price are excluded	Correct results displayed	Pass
TC-012	Apply price range	Enter both minimum and maximum price limits	Only courses within specified range are displayed	Correct results displayed	Pass
TC-013	Combine category and level	Select a specific category and level	Only courses matching both filters are displayed	Correct results displayed	Pass
TC-014	Combine search and level	Enter search text and select a level	Only courses satisfying both conditions are displayed	Correct results displayed	Pass
TC-015	Combine all filters	Apply search, category, level, min and max price	Only courses satisfying all active filters are displayed	Correct results displayed	Pass
TC-016	Result counter	Apply any filter criteria	Counter displays "Showing X of Y courses"	Correct count displayed	Pass
TC-017	Active filter chips	Apply one or more filters	Active filters render as styled chips	Correct chips displayed	Pass
TC-018	Clear All Filters	Apply multiple filters and click "Clear All Filters"	All filters reset and full catalogue is restored	Filters cleared successfully	Pass

Test ID	Test Case	Test Steps	Expected Result	Actual Result	Status
TC-019	No matching courses	Apply filters that match no courses in the catalogue	Displays: "No courses match your filters. Try changing or clearing your search criteria."	Correct message displayed	Pass
TC-020	No courses exist	Test when backend returns empty catalogue list	Displays: "There are no courses available at the moment."	Correct message displayed	Pass
TC-021	Null description	Search catalogue when course has null description	Application executes smoothly without crashing	Application remained stable	Pass
TC-022	Null duration	Search catalogue when course has null duration	Application executes smoothly without crashing	Application remained stable	Pass
TC-023	Public access	Open catalogue and apply filters while logged out	Catalogue and all filters operate without authentication	Feature worked correctly	Pass
TC-024	Regression test	Navigate course cards, view course details, check layout	All existing features function normally without regression	Existing functionality worked	Pass
TC-025	Cards responsive grid (auto-fill)	Filter catalogue down to 1 matching course card	Card retains standard width (minmax 270px) and does not stretch across full page	Card width stayed uniform and correctly proportioned	Pass
TC-026	Tablet responsive filter layout	Resize viewport to $\leq$ 1100px (tablet breakpoint)	Filter bar reorganizes into a 3-column grid layout	Layout reorganized cleanly	Pass
TC-027	Mobile responsive filter layout	Resize viewport to $\leq$ 700px (mobile breakpoint)	Filter bar switches to single column; Clear button becomes full width; chips stack	Responsive single column verified	Pass
TC-028	Clear button conditional render	Observe Clear button before and after applying filters	Hidden when all filters are default; visible when any filter is active	Button visibility toggles accurately	Pass

11. Acceptance Criteria Verification

Acceptance Criterion	Verification	Status
Searching for "backend" returns matching courses	Tested text search across title, description, and category	Passed
Selecting Advanced displays only Advanced courses	Tested Level dropdown = Advanced	Passed
Selecting category and level displays only matching courses	Tested combined category and level filters	Passed
Setting price range filters courses correctly	Tested minimum and maximum price boundary conditions	Passed
Clear All Filters restores original catalogue	Tested all filters followed by Clear All button	Passed
Null descriptions and durations do not cause errors	Tested catalogue with missing or null attributes	Passed
Page works for users who are not logged in	Tested public catalogue without authentication token	Passed
Course cards maintain uniform size and do not stretch with few results	Verified CSS auto-fill keeps card proportions consistent	Passed
Filters adapt responsively on tablet ($\leq 1100\text{px}$) and mobile ($\leq 700\text{px}$)	Tested layout behavior across mobile, tablet, and desktop viewports	Passed

12. Evidence

The following screenshots should be included as evidence of implementation and testing:

Evidence 1 – Original Course Catalogue

Show the public course catalogue displaying the full list of courses before applying filters.

Evidence 2 – Text Search

Show a search term entered into the search field and the resulting matching courses.

Evidence 3 – Level Filter

Show the Level dropdown with "Advanced" selected and only Advanced courses displayed.

Evidence 4 – Multiple Filters

Show category and level filters being used simultaneously.

Evidence 5 – Price Range

Show minimum and maximum price values entered and the resulting filtered courses.

Evidence 6 – Result Counter and Filter Chips

Show the result counter ("Showing X of Y courses") and active filter chips for applied criteria.

Evidence 7 – Clear All Filters

Show multiple filters applied, followed by the Clear All Filters action and immediate restoration of the full catalogue.

Evidence 8 – No Matching Courses Empty State

Show the message displayed when courses exist but no course matches the applied filter criteria.

Evidence 9 – Empty Catalogue Empty State

Show the message displayed when there are no courses loaded in the catalogue.

Evidence 10 – Null Data Handling

Show that a course with null description or duration renders and filters safely without error.

Evidence 11 – Responsive Course Card Grid (Auto-fill Fix)

Show the course grid filtered down to 1 or 2 courses, demonstrating that cards maintain standard width (minmax 270px) rather than over-stretching across the full container width.

Evidence 12 – Responsive Filter Controls on Tablet and Mobile

Show the filter bar adapting to 3 columns on tablet viewports ($\leq 1100\text{px}$) and stacking vertically with a full-width Clear button on mobile viewports ($\leq 700\text{px}$).

Note: Screenshots do not necessarily need to be provided for every individual test case. A single screenshot may be used as evidence for multiple related requirements where appropriate.

13. Change Verification

The completed implementation was reviewed against the original Change Request and technical requirements.

Requirement	Implemented
Text search across title, category, level, description, duration	Yes
Case-insensitive text search matching	Yes
Dynamic category dropdown filter	Yes
Level dropdown filter (All, Beginner, Intermediate, Advanced)	Yes
Minimum price numerical filter	Yes
Maximum price numerical filter	Yes
Strict combined AND filtering logic	Yes
Result counter ("Showing X of Y courses")	Yes
Active filter chips display	Yes
Clear All Filters button	Yes
Dynamic Clear All button visibility (only when filters active)	Yes
Reset search text, category, level, min and max price	Yes
Differentiated empty states (No courses vs. No matching results)	Yes
Frontend-only execution without redundant network calls	Yes
Course card grid responsive fix (auto-fill prevents stretching)	Yes
Tablet responsive filter layout (3 columns at $\leq 1100\text{px}$)	Yes
Mobile responsive filter layout (single column at $\leq 700\text{px}$)	Yes
Number input spinners suppressed in filter bar	Yes
Null-safe description handling	Yes
Null-safe duration handling	Yes
Public/logged-out catalogue access	Yes
Backend changes required	No
Database changes required	No
Additional API requests introduced	No

14. Final Change Status

Change Request ID: CR-001

Change Title: Advanced Search, Course Filtering Improvements, and Responsive Card Grid Fix

Implementation Status: Completed

Testing Status: Completed

Demonstration Status: Completed

Acceptance Criteria: Passed

Backend Changed: No

Database Changed: No

API Changed: No

Frontend Changed: Yes

Files Modified:

- frontend/src/pages/Courses.jsx
- frontend/src/App.css

Final Result

The public course catalogue now provides comprehensive, advanced search and filtering functionality alongside robust responsive design improvements. Users can search across course titles, categories, levels, descriptions, and durations; filter by category, level, and numeric price range; combine filters simultaneously using strict AND logic; view real-time result counters; inspect active filter chips; and reset all criteria with a single click using the dynamic Clear All Filters button.

In addition, key visual and responsive flaws were addressed. The course card grid layout was resolved by transitioning from auto-fit to auto-fill, ensuring that course cards maintain consistent, well-proportioned dimensions and do not over-stretch across the screen when few matching courses are returned. The filter bar was made fully responsive across desktop, tablet ($\leq 1100\text{px}$: 3-column layout), and mobile ($\leq 700\text{px}$: stacked single-column layout with full-width button and vertical filter chips).

The implementation is executed entirely on the frontend with zero backend, database, or API changes required. All potentially null, undefined, or malformed values (such as descriptions, durations, and prices) are handled safely with null-safe guards. Differentiated empty states clearly guide the user whether the catalogue itself is empty or their current filter combination matched no courses.

This version is structured so the student can use it as the actual CR implementation/report document, while the test table and evidence sections can be completed with their real screenshots and test results.